Why This Course, and Why Python

DSA27 — Lecture 01 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

22 September 2026

Before We Start

Where everything lives

What	Where
Code, exercises, these notes	https://github.com/helghareeb/DSA27
Course updates, announcements	WhatsApp channel
Some — not all — video lectures	https://www.youtube.com/@0xHGH

The one thing to do today

```
git clone https://github.com/helghareeb/DSA27.git
cd DSA27
py -3.13 -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
pytest -m "not challenge"
```

Every one of those tests must pass **before** you write a line of course code.

Programming Languages Are Not the Same

The argument that sounds right

Every language can print `Hello, World`.

Every language can add two numbers.

Every language is, in the end, reduced to instructions the same processor executes.

So all programming languages are the same. Only the syntax differs.

Why it is wrong

Turing-equivalence says: **the same problems are solvable.**

It does not say they are solvable:

- at the same **cost**
- by the same number of **people**
- in the same amount of **time**
- with the same number of **mistakes surviving to production**

A language is a set of decisions

Programming Language $\longrightarrow$ Design $\longrightarrow$ Philosophy

Ask of any language: **what did its designers refuse to do, and why?**

The decisions have owners

Person	Language	The decision, in one line
Dennis Ritchie	**C**	A portable assembler. Trust the programmer.
Bjarne Stroustrup	**C++**	Leave no room for a lower-level language.
Guido van Rossum	**Python**	Readability is worth paying for.
Graydon Hoare	**Rust**	Memory safety without a garbage collector.
Pike, Thompson, Griesemer	**Go**	Fewer features, faster builds, at scale.

Fifty years of decisions

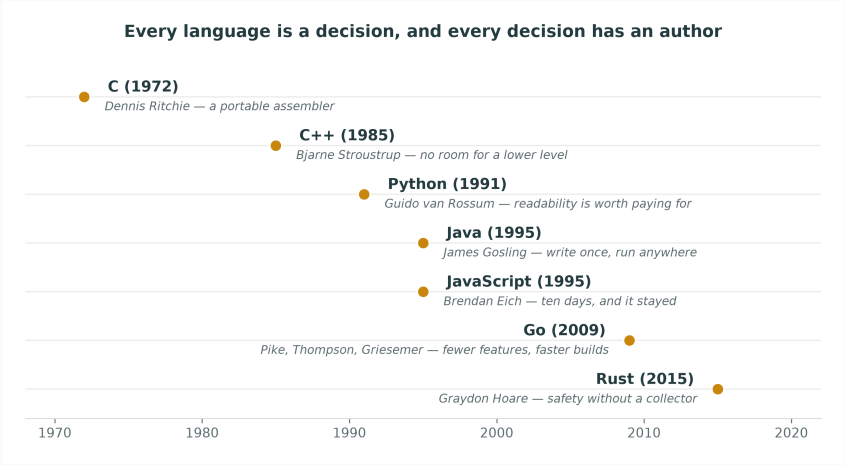


Figure: Each language answers a different question, and the answer has an author.

Where C came from

- **Dennis Ritchie**, at **Bell Labs**, in the early 1970s.
- Built out of Ken Thompson's language **B**, for one purpose: to write **Unix**.
- Unix was rewritten in C in 1973 — the first major operating system not written in assembly.
- Ritchie and Thompson shared the **1983 Turing Award**. Ritchie died in 2011.

The book to own

Brian W. Kernighan and Dennis M. Ritchie
The C Programming Language, 2nd edition
Prentice Hall, 1988 · ISBN 0-13-110362-8

Everyone calls it **K&R**. Get the **2nd edition** — it covers ANSI C.

https://cs.princeton.edu/~bwk/cbook.html

A correction I owe you

On the board I put **Donald Knuth** next to Stroustrup and van Rossum. That was loose, and I want to fix it.

- Knuth did not design a language you will write production code in.
- He wrote *The Art of Computer Programming*, created **TeX**, and invented **literate programming**.
- His subject is **algorithms** — their analysis, their correctness, their cost.

He is not the bridge to languages. He is the bridge to this course.

Paradigms: the same job, three ways

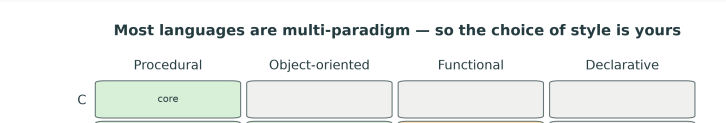
Sum the squares of the even numbers in a list.

```
# Procedural — describe the steps
total = 0
for n in numbers:
    if n % 2 == 0:
        total += n * n

# Functional — describe the transformation
total = sum(map(lambda n: n * n, filter(lambda n: n % 2 == 0, numbers)))

# Declarative (comprehension) — describe the result
total = sum(n * n for n in numbers if n % 2 == 0)
```

Same output. Same language. **Three different ways of thinking.**



Type systems: two axes, not one

Students collapse these into one question. They are independent.

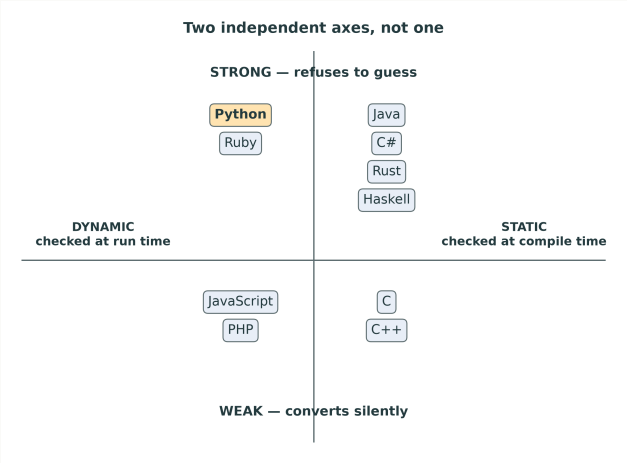
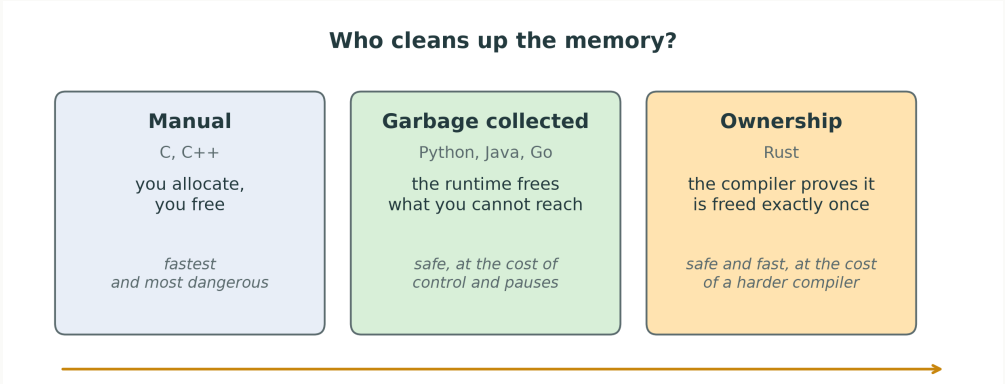


Figure: Static versus dynamic is *when*; strong versus weak is *how strictly*. Python sits top-left: dynamic, and strict.

Memory: who cleans up?

- **Manual** — you allocate, you free. C, C++. Fastest, and the source of decades of security vulnerabilities.
- **Garbage collected** — the runtime frees what you can no longer reach. Python, Java, C#, Go. Safe, at the cost of control and some pauses.
- **Ownership** — the compiler proves at build time that memory is freed exactly once. Rust. Safe *and* fast, at the cost of a harder compiler to satisfy.



Comparing Languages Honestly

What are you actually asking?

"Which language is best?" is not a question. These are questions:

- Best **for what task**? (a web service, embedded firmware, data analysis, a game)
- Best **for whom**? (you, alone, learning — or forty engineers for ten years)
- Best by **which measure**? (speed to write, speed to run, hiring pool, safety)
- Best **when**? (this project, or the one you maintain until 2035)

The rankings disagree — and that is the lesson

Index	What it actually counts	Crowns
TIOBE	Search-engine hits for "X programming"	Python
Stack Overflow Survey	Self-reported usage by respondents	JavaScript
GitHub / Octoverse	Commits and repositories	TypeScript / Python
PYPL	Google searches for "X tutorial"	Python

Different instruments. Different answers. **None of them measures quality.**

Ecosystem beats syntax

What you actually adopt when you adopt a language:

- **Libraries** — has someone already solved your problem, well?
- **Tooling** — debugger, profiler, package manager, formatter, test runner
- **Community** — is your question already answered?
- **Interoperability** — can it call the fast thing written in something else?
- **Longevity** — will it still be maintained in ten years?

Why Python for This Course

The honest case for Python

- **Where the field is.** Research, scientific computing, data analysis, and essentially all AI/ML/DL tooling.
- **Bindings.** The heavy numerical work is C, C++, CUDA or Fortran underneath; Python is the layer you steer it from. NumPy is not slow Python — it is fast C with a Python handle on it.
- **Reading cost.** Python is close enough to pseudocode that an algorithm survives the translation onto the page. That matters in a course where the algorithm *is* the content.
- **You will meet it again.** In every one of your programs — AI, Medical Informatics, Software Engineering.

The honest case against Python — for us

Python already contains, as built-ins, most of what this course studies.

You will build	Python already has
Dynamic array	`list`
Hash map	`dict`
Stack / queue	`list`, `collections.deque`
Sorting	`sorted()`, `list.sort()` — Timsort
Priority queue	`heapq`

The tool hides exactly the thing we are here to look at.

From a Variable to a Data Structure

The ladder

`variable` → `object` → `class` → `collection` → **data structure**

```
i = 42                      # a name bound to an object
s = "hello"                 # an object of class str
xs = [42, "hello", 3.14]    # a collection of references
```

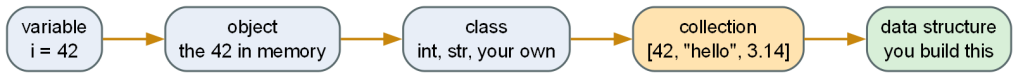


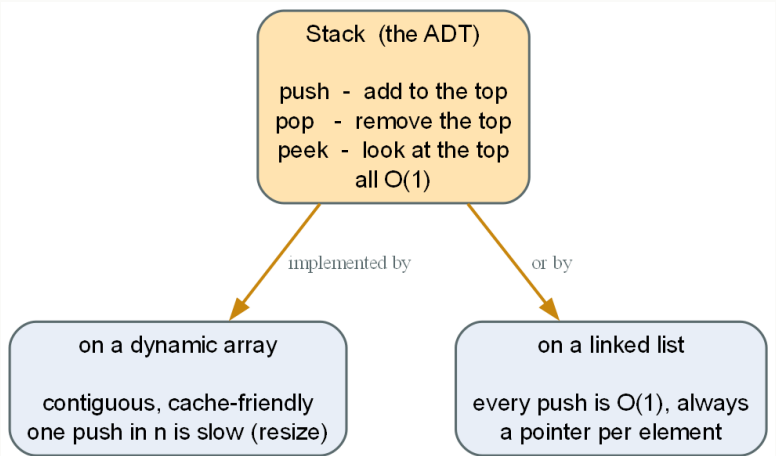
Figure: Five rungs. This course lives on the last one, but it only makes sense if the first four are solid.

In Python a variable is **not a box holding a value**. It is a **name bound to an object**.

Abstract Data Type vs implementation

Abstract Data Type (ADT) — النوع المجرد للبيانات The **contract**: what operations exist, what they mean, and what they cost. *Nothing* about how the data is stored.

Data structure — هيكل البيانات A concrete **arrangement of data in memory** that fulfils the contract.



Know-How

Three different things, and only the third is worth much:

1. **Know-what** — you can define a stack. *An hour's reading.*
2. **Know-how** — you can build one that works. *This course's exercises.*
3. **Know-when and know-why** — you can say why a stack and not a queue, and what it costs. *This course's exams, and your career.*

Cost Is the Whole Point

What an algorithm is

An **algorithm** is a finite, unambiguous procedure that turns an input into the correct output.

We care about three properties, in this order:

1. **Correctness** — does it produce the right answer, always?
2. **Cost in time** — how does the work grow as the input grows?
3. **Cost in space** — how does the memory grow as the input grows?

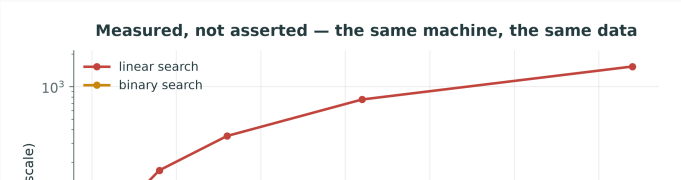
Measure, do not assert

Do not take my word that binary search beats linear search. **Measure it.**

```
from viz.complexity import measure, plot_growth
import random

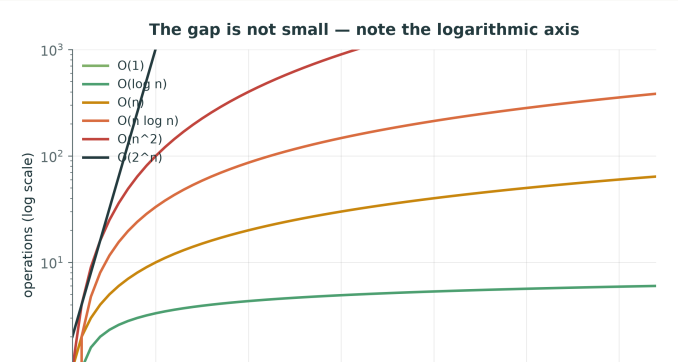
sizes = [1000, 2000, 4000, 8000, 16000]
make = lambda n: (sorted(random.random() for _ in range(n)), 0.5)

plot_growth(
    {"linear": measure(lambda a: linear_search(*a), sizes, make),
     "binary": measure(lambda a: binary_search(*a), sizes, make)},
    reference=["n", "log n"],
)
```



The gap is not small

n	O(log n)	O(n)	O(n log n)	$O(n^2)$	$O(2^n)$
10	3	10	33	100	1,024
100	7	100	664	10,000	$\sim 1.3 \times 10^{30}$
1,000	10	1,000	9,966	1,000,000	—
1,000,000	20	10^6	$\sim 2 \times 10^7$	10^{12}	—



The Course Itself

Your course, officially

Three programs, three bylaws, **one course**.

	AI	Bio / Medical Informatics	Software Engineering
Code	**CS2101**	**IS122**	**IS122**
Hours	3 (2 lec + 2 lab)	3 (2 lec + 2 lab)	3 (2 lec + 2 lab)
Prerequisite	CS1002 OOP	CS012, MATH012	CS012, MATH012
Level	Level 2, Sem. 3	Level 2	Sophomore, Sem. 1
Bylaw	2020, p. 44	2014, p. 35	2013, p. 38

Different codes. Same subject, same hours, same room.

How you pass

Same three rules in all three لائح:

- Final written exam is **60%**; coursework is the other **40%**.
- Pass needs **≥ 60% overall and ≥ 30% of the final** — both.
- **≥ 75% attendance**, or you are محروم and cannot sit the final at all.

How we will work

- **Lecture** — the idea, the cost, and why it exists.
- **Lab** — you implement it. The skeletons in `dsa/` all raise `NotImplementedError` on day one. That is the assignment.
- **Tests** — `tests/` defines "correct". `pytest -m challenge` runs every exercise.
- **Visualisation** — `viz/` draws arrays, lists, trees and graphs, animates your sorts, and plots your measured complexity. It never imports `dsa/`, so it works with whatever you write.

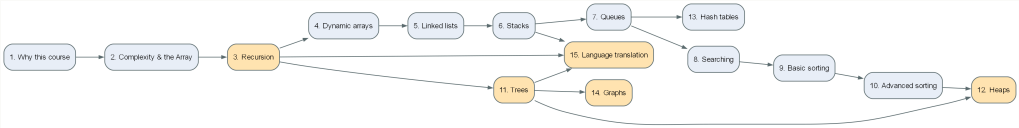


Figure: Fifteen weeks. An arrow means the later week genuinely needs the earlier one — which is why recursion comes before trees, and trees before the parser.

Homework 1

Due before the next lecture

1. **Set up your environment.** Clone the repository, create the virtual environment, install the requirements, and make `pytest -m "not challenge"` pass. Bring the output.
2. **Read** Peter Norvig, *Teach Yourself Programming in Ten Years* — https://norvig.com/21-days.html. It is short.
3. **Run** `import this` in a Python prompt and read *The Zen of Python*. Pick the line you disagree with most, and be ready to say why.
4. **Write — one page, no more.** Choose two languages you have used or heard about. Compare them on **three criteria you choose yourself**, say which you would pick for a **specific named task**, and state what you are giving up by picking it.

Practise online

- **Week 1 quiz**: 42 multiple-choice questions, every answer explained
- **Written practice** on the same page: 10 questions. Write, then mark yourself against the marking points
- **Lecture 1 test**: 20 questions, 25 minutes, timed
- **Midterm simulator**: Lectures 1–7, papers of up to 60 questions
- In Arabic: week 1 · midterm

Summary

Six things to keep

1. Languages are **not** the same. Each is a set of decisions with an author and a reason.
2. "Which is best?" is unanswerable until you say **best for what, for whom, by which measure**.
3. Popularity indices measure **different things** and disagree. None measures quality. Always check the date.
4. We use Python because it reads like the algorithm — and we **reimplement its built-ins** because it otherwise hides the subject.
5. An **ADT** is a contract; a **data structure** is one way to keep it. Choosing between them is the engineering.
6. **Know-how** beats know-what; **know-why** beats both.

Next

Week 2 — Complexity. How to say what an algorithm costs, and how to *measure* whether you were right.

Before then: make `pytest -m "not challenge"` pass.